39

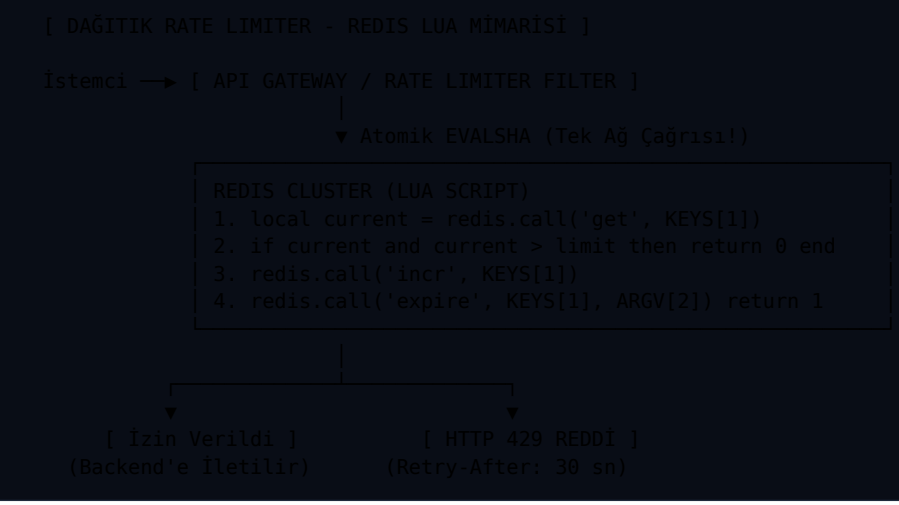
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Dağıtık Hız Sınırlayıcı (Distributed Rate Limiter) Tasarımı: Redis Lua ve Multi-Region

Designing a Distributed Rate Limiter: Redis Lua & Multi-Region

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir gece kulübünün kapısındaki fedaiyi düşünün (Rate Limiter). Kulübün kuralı: 'Bir kişi dakikada en fazla 5 kez içeri girip çıkabilir'. Fedainin elinde telsizle bağlı olduğu merkezi bir liste (Redis) vardır. Bir kişi kapıya geldiğinde fedai listeye bakar, sayacı 1 artırır. Eğer 5 hakkını doldurduysa 'Dur bakalım, 1 dakika sokakta bekle!' der (HTTP 429). Lua Script ise fedainin listeye bakıp sayıyı artırma işini göz açıp kapayıncaya kadar tek bir saniyede (Atomik olarak) yapmasıdır; böylece iki kişi aynı anda kapıya yüklense bile fedaiyi kandıramaz!

🇬🇧 ENGLISH (ELI5)

Picture a VIP nightclub bouncer guarding the velvet rope (Rate Limiter). House rule: 'No patron can enter more than 5 times per minute'. The bouncer holds a radio linked to the central clipboard (Redis). When a patron approaches, the bouncer checks the tally, increments the counter, and if the count exceeds 5, blocks entry with 'Wait 60 seconds on the curb!' (HTTP 429). The Redis Lua script ensures the bouncer checks and increments the clipboard atomically in a single nanosecond, preventing two patrons from rushing the rope simultaneously!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Dağıtık Hız Sınırlayıcı Mimarisi: 1) Yarış Durumu (Race Condition) Çözümü: Eğer uygulama Redis'ten `GET` yapıp kodda artırıp `SET` yaparsa, eşzamanlı iki istek aynı değeri okur ve limit aşılır. Çözüm: Redis Lua Scripting. Lua betikleri Redis motorunda tek iş parçacığında atomik olarak çalışır; sıfır kilit maliyetiyle `GET + INCR + EXPIRE` tek adımda biter. 2) Çok Bölgeli (Multi-Region) Senkronizasyon: Kullanıcı hem ABD hem Avrupa bölgesinden istek atıyorsa ne olur? Çözüm: A) Merkezi Redis (Ağ gecikmesi ekler). B) Yerel Redis + Asenkron Sync (CRDT / Eventual Consistency): Her bölgeye kotanın bir kısmı tahsis edilir (Local Quota Allocation) veya asenkron eşleme yapılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Distributed Rate Limiter Architecture: 1) Atomic Race Condition Elimination: Executing a separate `GET`, incrementing in JVM memory, and writing back via `SET` introduces fatal concurrency race windows. Resolution: Redis Lua Scripting executes `GET`, conditional comparison, `INCR`, and `EXPIRE` atomically within the Redis single-threaded engine in a single network round-trip. 2) Multi-Region Synchronization: If a user fires requests simultaneously across US-East and EU-West, cross-datacenter synchronization is required. Architecture Options: A) Centralized primary Redis cluster (adds 100ms cross-Atlantic latency). B) Localized Quota Splitting: Partition a user's 100 req/min allowance into 50 for US and 50 for EU, synchronizing background replenishment asynchronously via batch consensus.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Redis Lua Script Tabanlı Rate Limiting Mantığı Simülasyonu
public class RedisLuaRateLimiter {
    public static final String LUA_SCRIPT = """
        local key = KEYS[1]
        local limit = tonumber(ARGV[1])
        local window = tonumber(ARGV[2])

        local current = redis.call('get', key)
        if current and tonumber(current) >= limit then
            return 0 -- İzin Verilmedi (HTTP 429)
        else
            redis.call('incr', key)
            if not current then
                redis.call('expire', key, window)
            end
            return 1 -- İzin Verildi (HTTP 200)
        end
        """;

    public record RateLimitResult(boolean allowed, int remainingLimit, int retryAfterSeconds) {}
}

void main() {
    IO.println("Redis Lua Script Hazırlandı (Atomik GET + INCR + EXPIRE)");
    IO.println("Script Boyutu: " + RedisLuaRateLimiter.LUA_SCRIPT.trim().length() + " karakter");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta rate limiting için istemci IP adresini tek başına anahtar (Key) olarak seçmeyin! Aynı kurumsal ofisteki veya üniversitedeki 10.000 kişi aynı NAT Gateway IP'sini paylaşabilir; tek bir kullanıcının kotası dolunca tüm şirket bloklanır. Anahtar: `UserId` (Giriş yapmışsa) veya `UserId + IP` olmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never rely solely on client IP for rate limiting! Behind enterprise NAT proxies or university campuses, 10,000 distinct employees share a single public IP. Key by authenticated `UserId` or a combination of `DeviceFingerprint + IP`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Redis Lua Scripting yarış durumlarını (Race Conditions) sıfır kilit maliyetiyle çözer.
- Kullanıcı deneyimi için HTTP 429 yanıtı ile birlikte 'Retry-After' başlığı dönülmelidir.
- Çok bölgeli sistemlerde yerel kota tahsisi (Local Quota Splitting) ağ gecikmesini engeller.

🎯 Key Takeaways for the Interview

- Redis Lua scripts execute check-and-increment operations atomically without distributed locks.
- Always accompany HTTP 429 responses with accurate Retry-After and X-RateLimit headers.
- Local quota partitioning eliminates cross-datacenter WAN latency on edge rate checks.